

Lenguajes y Compiladores — Apunte para el Final

Resumen de Reynolds, *Theories of Programming Languages* (2009)

Nicolás Cagliero

28 de junio de 2026

Índice general

1. Dominios, punto fijo y lenguaje imperativo	1
1.1. El lenguaje imperativo simple (Reynolds §2.1–2.2)	1
2. Cálculo lambda	4
3. Lenguajes aplicativos e Iswim	5

Capítulo 1

Dominios, punto fijo y lenguaje imperativo

1.1. El lenguaje imperativo simple (Reynolds §2.1–2.2)

Motivación

Queremos darle un *significado* matemático preciso a un lenguaje imperativo. Las cuatro primeras construcciones salen directo; el **while** es el que obliga a inventar toda la maquinaria de dominios y punto fijo (§2.3–2.4). Este capítulo deja planteado ese problema.

Sintaxis

$$\begin{aligned}\langle \textit{intexp} \rangle &::= 0 \mid 1 \mid 2 \mid \dots \mid \langle \textit{var} \rangle \mid -\langle \textit{intexp} \rangle \mid \langle \textit{intexp} \rangle + \langle \textit{intexp} \rangle \mid \dots \mid \langle \textit{intexp} \rangle \mathbf{rem} \langle \textit{intexp} \rangle \\ \langle \textit{boolexp} \rangle &::= \mathbf{true} \mid \mathbf{false} \mid \langle \textit{intexp} \rangle = \langle \textit{intexp} \rangle \mid \langle \textit{intexp} \rangle < \langle \textit{intexp} \rangle \mid \dots \mid \neg \langle \textit{boolexp} \rangle \mid \langle \textit{boolexp} \rangle \wedge \langle \textit{boolexp} \rangle \\ \langle \textit{comm} \rangle &::= \langle \textit{var} \rangle := \langle \textit{intexp} \rangle \mid \mathbf{skip} \mid \langle \textit{comm} \rangle ; \langle \textit{comm} \rangle \\ &\quad \mid \mathbf{if} \langle \textit{boolexp} \rangle \mathbf{then} \langle \textit{comm} \rangle \mathbf{else} \langle \textit{comm} \rangle \mid \mathbf{while} \langle \textit{boolexp} \rangle \mathbf{do} \langle \textit{comm} \rangle\end{aligned}$$

Todas las variables son enteras (no hay variables booleanas).

Estados

Definición 1.1 (Estado y actualización). Un *estado* $\sigma \in \Sigma$ asigna a cada variable un entero. La *actualización* $[\sigma \mid v : n]$ es el estado igual a σ salvo que manda $v \mapsto n$:

$$[\sigma \mid v : n](w) = \begin{cases} n & \text{si } w = v \\ \sigma(w) & \text{si } w \neq v. \end{cases}$$

Semántica de expresiones

$$\llbracket \cdot \rrbracket_{\text{intexp}} : \langle \text{intexp} \rangle \rightarrow (\Sigma \rightarrow \mathbb{Z}), \quad \llbracket \cdot \rrbracket_{\text{boolexp}} : \langle \text{boolexp} \rangle \rightarrow (\Sigma \rightarrow \mathbb{B}).$$

Hecho clave: las expresiones *siempre terminan* (sin error).

Semántica de comandos

Un comando puede no terminar. Introducimos $\perp$ (*bottom*) para la no-terminación y $\Sigma_{\perp} = \Sigma \cup \{\perp\}$. Entonces

$$\llbracket \cdot \rrbracket_{\text{comm}} : \langle \text{comm} \rangle \rightarrow (\Sigma \rightarrow \Sigma_{\perp}).$$

Definición 1.2 (Extensión estricta). Si $f : \Sigma \rightarrow \Sigma_{\perp}$, su extensión estricta $f_{\perp} : \Sigma_{\perp} \rightarrow \Sigma_{\perp}$ es $f_{\perp}(\sigma) = \perp$ si $\sigma = \perp$, y $f(\sigma)$ si no.

Ecuaciones semánticas (dirigidas por la sintaxis):

$$\begin{aligned} \llbracket v := e \rrbracket_{\text{comm}} \sigma &= [\sigma \mid v : \llbracket e \rrbracket_{\text{intexp}} \sigma] \\ \llbracket \text{skip} \rrbracket_{\text{comm}} \sigma &= \sigma \\ \llbracket c_0 ; c_1 \rrbracket_{\text{comm}} \sigma &= (\llbracket c_1 \rrbracket_{\text{comm}})_{\perp} (\llbracket c_0 \rrbracket_{\text{comm}} \sigma) \\ \llbracket \text{if } b \text{ then } c_0 \text{ else } c_1 \rrbracket_{\text{comm}} \sigma &= \text{if } \llbracket b \rrbracket_{\text{boolexp}} \sigma \text{ then } \llbracket c_0 \rrbracket_{\text{comm}} \sigma \text{ else } \llbracket c_1 \rrbracket_{\text{comm}} \sigma \end{aligned}$$

Ejemplo resuelto

$$\begin{aligned} \llbracket x := x - 1 ; y := y + x \rrbracket_{\text{comm}} \sigma &= (\llbracket y := y + x \rrbracket_{\text{comm}})_{\perp} (\llbracket x := x - 1 \rrbracket_{\text{comm}} \sigma) \\ &= (\llbracket y := y + x \rrbracket_{\text{comm}})_{\perp} [\sigma \mid x : \sigma x - 1] \quad (\neq \perp, \text{ así que aplica } \llbracket y := y + x \rrbracket_{\text{comm}}) \\ &= [\sigma \mid x : \sigma x - 1 \mid y : \sigma y + (\sigma x - 1)] \\ &= [\sigma \mid x : \sigma x - 1 \mid y : \sigma y + \sigma x - 1]. \end{aligned}$$

Conexiones y trampas

El **while** no se puede definir así. Desenrollando, **while** b **do** $c \equiv \text{if } b \text{ then } (c ; \text{while } b \text{ do } c) \text{ else skip}$, de donde sale la *ecuación de desenrollado*

$$\llbracket \text{while } b \text{ do } c \rrbracket_{\text{comm}} \sigma = \text{if } \llbracket b \rrbracket_{\text{boolexp}} \sigma \text{ then } (\llbracket \text{while } b \text{ do } c \rrbracket_{\text{comm}})_{\perp} (\llbracket c \rrbracket_{\text{comm}} \sigma) \text{ else } \sigma. \quad (2.2)$$

No es una ecuación semántica válida: no está dirigida por la sintaxis (aparece **while** b **do** c a la derecha). Es una *condición*, y puede tener cero, una o varias soluciones.

- **while** $x \neq 0$ **do** $x := x - 2$: para x impar o negativo (no termina) la ecuación deja el resultado libre $\Rightarrow$ varias soluciones.
- **while true do skip**: *toda* función $\Sigma \rightarrow \Sigma_{\perp}$ la satisface.

Gancho a §2.3: hace falta ordenar los significados por “grado de definición” ($\sqsubseteq$) y tomar el *menor punto fijo* del funcional de desenrollado. Para eso: *dominios y funciones continuas*.

Capítulo 2

Cálculo lambda

Capítulo 3

Lenguajes aplicativos e Iswim